Министерство транспорта Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования «Российский университет транспорта»

(РУТ (МИИТ))

Институт железнодорожного транспорта

Кафедра «Цифровые технологии управления транспортными процессами»

Практическая работа №3

По предмету «Сбор, хранение и обработка больших данных»

На тему «Реализация алгоритма подсчета слов (WordCount) с использованием MapReduce в Hadoop»

Выполнил: Груздева К. А.

Группа: ЦИИ-171

Проверил: Кухтинов А. Э.

Москва 2026 г.

Оглавление

1. Цель и задачи работы 3

2. Ход выполнения 3

3. Ответы на контрольные вопросы 8

4. Выводы 9

1. Цель и задачи работы

Цель работы: реализовать алгоритм WordCount с использованием модели MapReduce в Hadoop, освоив полный цикл разработки распределенного приложения: от подготовки данных до анализа результатов.

Задачи:

1. Подготовить тестовые данные в HDFS.
2. Реализовать Mapper, Reducer, Combiner и Driver-класс на Java.
3. Собрать проект в JAR-файл с помощью Maven.
4. Запустить задание в кластере Hadoop.
5. Проанализировать результаты и производительность.

2. Ход выполнения

Для начала я создала в HDFS структуру директорий для проекта, а затем на локальной машине подготовила три тестовых текстовых файла с разным содержимым. Все три файла были загружены в HDFS в директорию input. Команда hdfs dfs -ls подтвердила, что файлы успешно загружены.

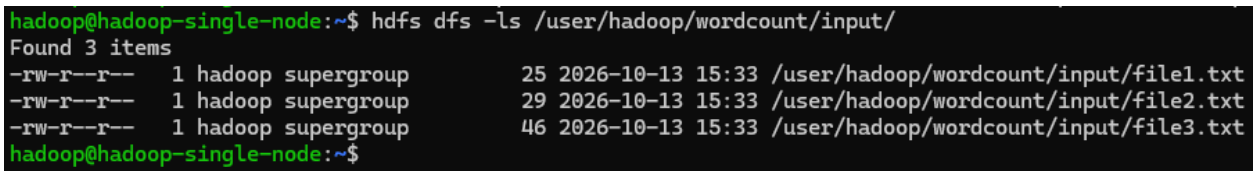


Рисунок 1 – Загруженные файлы в HDFS

После подготовки данных я создала новый Maven-проект в IntelliJ IDEA под названием wordcount. В файл pom.xml я добавила зависимости от библиотек Hadoop версии 3.3.6, а также указала, что компиляция должна производиться под Java 8, так как эта версия лучше всего совместима с Hadoop.

pom.xml (wordcount)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.datatech</groupId>
    <artifactId>wordcount</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>8</maven.compiler.source>
        <maven.compiler.target>8</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.apache.hadoop</groupId>
            <artifactId>hadoop-client</artifactId>
            <version>3.3.6</version>
        </dependency>
        <dependency>
            <groupId>org.apache.hadoop</groupId>
            <artifactId>hadoop-hdfs</artifactId>
            <version>3.3.6</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
                <configuration>
                    <source>8</source>
                    <target>8</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Рисунок 2 – Файл pom.xml с зависимостями Hadoop

Затем я реализовала четыре класса в пакете com.datatech.wordcount. Класс WordCountMapper разбивает входные строки на слова, очищает их от знаков препинания и приводит к нижнему регистру, после чего генерирует промежуточные пары вида слово и единица. Класс WordCountReducer суммирует все единицы для каждого слова и выдает финальную пару слово и общее количество. Класс WordCountCombiner выполняет предварительную

агрегацию на стороне Mapper'a, уменьшая объем данных, передаваемых по сети, и использует ту же логику, что и Reducer. Класс WordCountDriver координирует выполнение всего задания: настраивает конфигурацию, определяет используемые классы и задает входные и выходные пути.

```java
package com.datatech.wordcount;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCountDriver {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: WordCount <input path> <output path>");
            System.exit( status: -1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf,  jobName: "word count");

        job.setJarByClass(WordCountDriver.class);
        job.setMapperClass(WordCountMapper.class);
        job.setCombinerClass(WordCountCombiner.class);
        job.setReducerClass(WordCountReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion( verbose: true) ? 0 : 1);
    }
}
```

Рисунок 3 – Класс WordCountDriver (координация задания)

С помощью Maven я собрала проект в JAR-файл. Команда mvn clean package завершилась успешно, и в директории target появился файл wordcount-1.0-SNAPSHOT.jar.

```
PS C:\Users\User\Documents\wordcount> mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] -----------------------< com.datatech:wordcount >-----------------------
[INFO] Building wordcount 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[WARNING] 2 problems were encountered while building the effective model for org.apache.yetus:audience-annotations:jar:0.5.0 during dependency collection step for project (use -X to see details)
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ wordcount ---
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ wordcount ---
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ wordcount ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 5 source files with javac [debug target 8] to target\classes
[WARNING] bootstrap class path not set in conjunction with -source 8
[WARNING] source value 8 is obsolete and will be removed in a future release
[WARNING] target value 8 is obsolete and will be removed in a future release
[WARNING] To suppress warnings about obsolete options, use -Xlint:-options.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ wordcount ---
[INFO] skip non existing resourceDirectory C:\Users\User\Documents\wordcount\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ wordcount ---
[INFO] Changes detected - recompiling the module! :dependency
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ wordcount ---
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ wordcount ---
[INFO] Building jar: C:\Users\User\Documents\wordcount\target\wordcount-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.012 s
[INFO] Finished at: 2026-10-13T18:51:21+03:00
[INFO] ------------------------------------------------------------------------
```

Рисунок 4 – Успешная сборка проекта (BUILD SUCCESS)

С помощью утилиты scp я скопировала собранный JAR-файл с Windows на виртуальную машину, где запущен кластер Hadoop. Копирование прошло успешно.

```
C:\Users\User>scp C:\Users\User\Documents\wordcount\target\wordcount-1.0-SNAPSHOT.jar hadoop@192.168.3.22:~/
hadoop@192.168.3.22's password:
wordcount-1.0-SNAPSHOT.jar                                    100% 7119     2.3MB/s   00:00
```

Рисунок 5 – Копирование JAR-файла на виртуальную машину через scp

После этого я запустила задание MapReduce на кластере. В процессе выполнения я наблюдала за прогрессом Map и Reduce задач. Задание успешно завершилось с сообщением Job completed successfully. Из логов видно, что было запущено три Map-задачи, по одной на каждый файл, и одна Reduce-задача. Combiner уменьшил количество промежуточных записей с пятнадцати до четырнадцати, что снизило объем данных, передаваемых по сети.

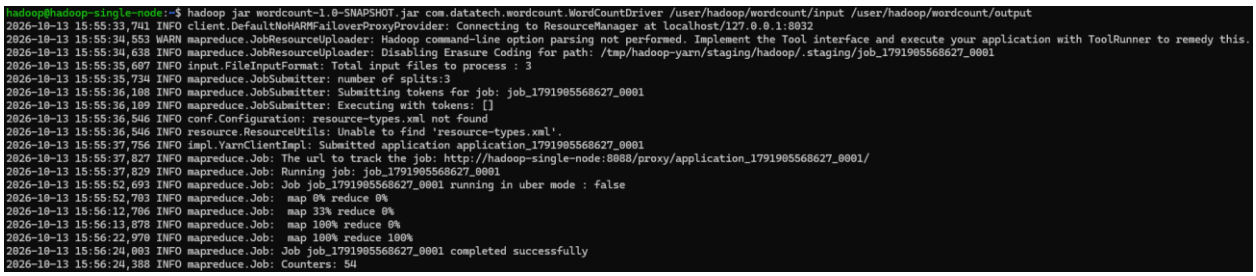


Рисунок 6 – Успешное завершение задания MapReduce (Job completed successfully)

Затем я проверила содержимое выходной директории и просмотрела файл результатов part-r-00000. Результат подсчета слов полностью совпал с ожидаемым: слово hadoop встретилось три раза, hello тоже три раза, mapreduce два раза, world два раза, а остальные слова по одному разу.



Рисунок 7 – Результат подсчета слов (part-r-00000)

В завершение я открыла веб-интерфейс ResourceManager по адресу http://192.168.3.22:8088 и нашла свое задание в списке приложений. На странице задания отображается подробная информация: статус SUCCEEDED, время запуска и завершения, количество Map и Reduce задач, а также различные метрики производительности.

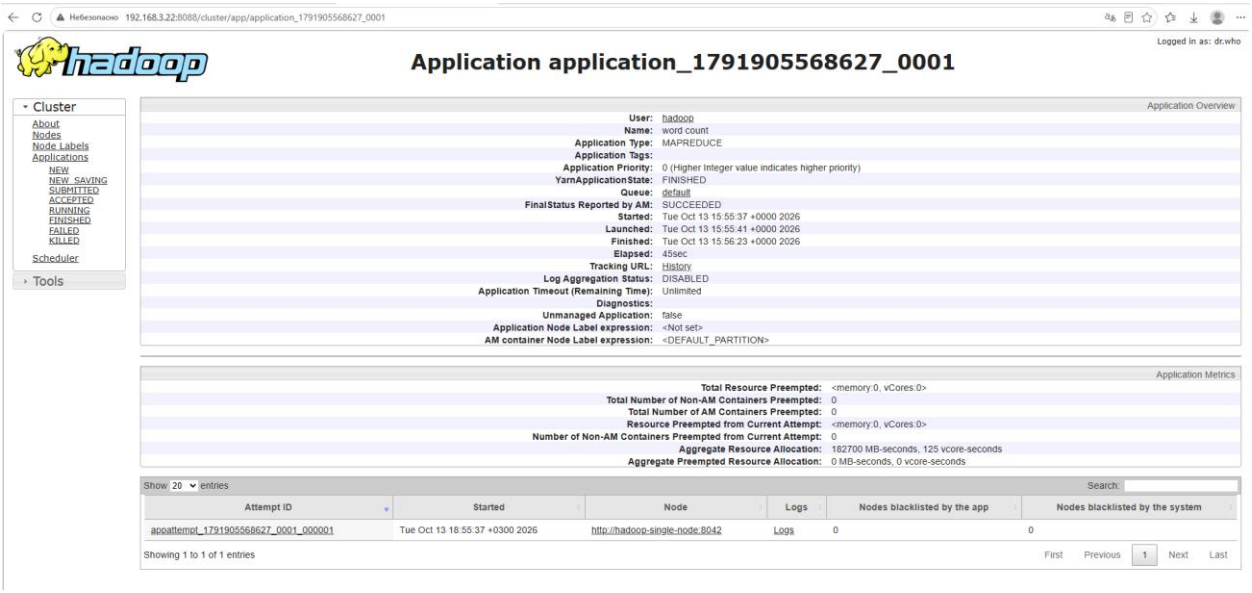


Рисунок 8 – Страница задания в веб-интерфейсе ResourceManager

3. Ответы на контрольные вопросы

1. Увеличение количества Reducer'ов позволяет распараллелить фазу Reduce, что ускоряет обработку больших объемов данных. Однако слишком большое количество Reducer'ов может привести к накладным расходам на координацию и созданию множества мелких файлов, что снижает общую эффективность.

2. Combiner можно использовать только для коммутативных и ассоциативных операций, таких как суммирование или поиск максимума. Для операций, зависящих от порядка, например вычисления среднего арифметического, Combiner может дать неверный результат, так как он применяется к части данных, а не ко всему набору.

3. Отказоустойчивость обеспечивается за счет того, что Application Master отслеживает выполнение задач через механизм Heartbeat. Если задача падает или зависает, AM перезапускает её на другом узле. Промежуточные данные Mapper'ов хранятся на локальном диске, и если Mapper упал, его данные пересчитываются.

4. Application Master обнаружит это по отсутствию Heartbeat или по сообщению об ошибке, и задача будет перезапущена на другом узле. Если ошибка повторяется несколько раз, задание может быть помечено как FAILED.

5. Улучшить алгоритм можно несколькими способами. Например, можно игнорировать стоп-слова, такие как the, and, or, которые не несут смысловой нагрузки. Можно также подсчитывать N-граммы вместо отдельных слов или фильтровать слова по минимальной частоте встречаемости. Еще один вариант — использовать более эффективную очистку текста, например, с помощью библиотеки Apache Lucene.

4. Выводы

В ходе работы я освоила полный цикл разработки MapReduce-приложения: от написания кода на Java до запуска задания на кластере Hadoop. Я реализовала алгоритм WordCount с использованием Mapper, Reducer, Combiner и Driver, собрала проект в JAR-файл с помощью Maven и запустила его на кластере. Я проанализировала результаты и метрики производительности через веб-интерфейс ResourceManager. Использование Combiner'a позволило уменьшить объем данных, передаваемых по сети, что повысило производительность. Основная сложность возникла при настройке зависимостей Maven и обеспечении совместимости с Java 8, но это помогло глубже понять структуру MapReduce-приложения. Работа с MapReduce показалась мне логичной и наглядной, особенно в части параллельной обработки данных.